Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Webová aplikace s texty Bible

2026

Patrik Prinz

Vedoucí práce:
Mgr. Tomáš Urbanec, Ph.D.

Studijní program: Informatika,
Specializace: Programování a vývoj software

Bibliografické údaje

Autor:	Patrik Prinz
Název práce:	Webová aplikace s texty Bible
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet stran:	13
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	český

Bibliographic info

Author:	Patrik Prinz
Title:	Web application with Bible texts
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	13
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Czech

Anotace

Ukázkový text závěrečné práce na Katedře informatiky Přírodovědecké fakulty Univerzity Palackého v Olomouci, který je zároveň dokumentací stylu pro text práce v LaTeXu. Zdrojový text v LaTeXu je doporučeno použít jako šablonu pro text skutečné závěrečné práce studenta.

Synopsis

Sample text of thesis at the Department of Computer Science, Faculty of Science, Palacký University Olomouc and, at the same time, documentation of the LaTeX style for the text. The source text in LaTeX is recommended to be used as a template for real student's thesis text.

Klíčová slova: webová aplikace; Bible; PWA; TypeScript; DevOps; CI/CD

Keywords: web application; Bible; PWA; TypeScript; DevOps; CI/CD

Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1 Úvod **7**

2 Podobné projekty **8**
2.1 Časoslov.sk 8
2.2 BibleGateway 8
2.3 Beblia.com 8

3 Návrh a struktura **8**
3.1 Balíčky 9
3.1.1 Kořenový balíček 9
3.1.2 Backend 9
3.1.2.1 Struktura modulu 9
3.1.3 Frontend 10
3.1.4 Databáze 10
3.2 Nastavení projektu 10
3.3 Verzování 10
3.3.1 Větve 10
3.3.2 Commity 11
3.4 CI/CD pipeline 11
3.5 Styl kódu 11

4 Použité technologie **11**
4.1 TypeScript 11
4.2 Express 12
4.3 Elastic search 12
4.4 Docker 12
4.5 Github actions 12

Seznam zkratek **13**

Seznam tabulek

1 Úvod

Cílem této práce je vytvořit aplikaci pro použití v Řeckokatolické farnosti v Olomouci. Farnost je v současnosti vícenárodnostní a v posledních letech se značně rozrůstá. nejpočetněji jsou zde zastoupeni farníci z Ukrajiny. Texty Bible jsou v katolické církvi velmi často využívané při liturgických obřadech, ale také při osobní modlitbě a rozjímání.

Pro Apoštolský Exarchát České Republiky, který je nadřazenou organizací Olomoucké farnosti existuje kalendář, který obsahuje i biblické texty pro každý den v roce. Biblické texty v církevních obřadech jsou často vázány ke svátkům, které jsou buď pevně svázány s datem, nebo jsou pohyblivé a připadají každý rok na jiné datum. Mnoho textů je také svázáno s konkrétním obřadem, nebo událostí (např. křest, svěcení domu...).

Texty z knihy žalmů jsou často používány jako samostatné modlitby. Východní církve žalmy rozdělují do 20 skupin zvaných kaftizmy, které obsahují většinou okolo 4-8 žalmů. Ke kaftizmám jsou přidruženy další texty, které se modlí zároveň s nimi.

Farnost má vlastní společenství, které se modlí modlitby žalmů a funguje od roku 2024. Každý člen společenství se modlí jednu z kaftizem během týdne. Kaftizmy se mezi členy společenství pravidelně střídají.

Nenašel jsem dosud žádný projekt, který by poskytoval možnosti práce s biblickými texty v tomto rozsahu. Proto chci vytvořit aplikaci, která by sdružovala texty více jazyků s možností vlastní správy a přidávání dalších překladů a textů pro vlastní četbu a studium. S tímto bude spojena i možnost udržování vlastního kalendáře který bude obsahovat zejména úryvky na každý den. Také by zde měla být obsažena možnost vytvářet si a ukládat vlastní úryvky pro vlastní použití, nebo sdílení ve skupině. Proces rozdělení kaftizem v rámci společenství žaltáře plánuji zčásti zautomatizovat a zároveň poskytnout na stejném místě i texty samotné i s přidruženými modlitbami spolu s možností označit, že se člen již kaftizmu pomodlil.

Dále bych chtěl, aby platforma byla do budoucna dále rozšiřitelná a bylo možné přidávat další funkcionality a aby mohl projekt i v budoucnu reflektovat požadavky společenství, které jej bude využívat a pravděpodobně se i dále rozvíjet.

Dále se chci v této práci zaměřit na průzkum a využití konceptů z oblasti DevOps, které usnadní vývoj samotný v dlouhodobém měřítku, urychlí integraci a nasazení aplikace. Na projektu pracuji sám, proto je pro mne výhodné především zautomatizovat proces ověření a nasazení software a sledování jeho stavu. I za cenu větší náročnosti přípravy a počáteční konfigurace je pro mne výhodné získat značně větší spolehlivost a rychlost nasazení a také do budoucna usnadní řadu problémů s udržováním potřebné infrastruktury. Díky této filosofii se mohu i v jednočlenném týmu, po nasazení potřebné infrastruktury, soustředit na svou roli programátora bez nutnosti stále ručně udržovat vše potřebné pro chod projektu.

2 Podobné projekty

V rámci této práce jsem hledal a procházel i další software zaměřený na práci s texty Bible i dalšími liturgickými texty. Aplikací s tímto zaměřeních existuje celá řada. Většina je však určena k jiným účelům a především pro osobní použití, ne pro použití ve farnosti samotné, případně v jiném společenství. Podobné projekty bývají zaměřeny na samotné čtení a studium biblických textů. Většina z projektů má také uzavřený zdrojový kód a proto je nelze upravit a přizpůsobit pro specifické potřeby, nebo je již zastaralá a dlouhodobě neudržovaná. Dále existuje software zaměřený na texty modliteb samotné. Ty však většinou obsahují pouze samotné modlitby, typicky zobrazované podle dne. Projekt, který vyvíjím v rámci této práce sdružuje a implementuje více specifických požadavků, které ve většině případů nespadají do stejné domény. Pro případ chodu farnosti, nebo jiného společenství je však výhodné je sdružit do jednoho projektu. V seznamu níže uvádím některé z projektů, se kterými jsem se dosud setkal.

2.1 Časoslov.sk

Časoslov.sk je projekt vyvíjený Radou pre mládež Košickej eparchie na slovensku. Poskytuje texty pro jednotlivé dny v roce. Nabízí webovou i mobilní aplikaci s obdobnými funkcionalitami. Časoslov.sk se zaměřuje především na samotné texty pro jednotlivé dny. Neposkytuje možnost samostatného čtení a studia biblických textů, správu více překladů, ani funkcionalitu skupin.

2.2 BibleGateway

BibleGateway je Webová aplikace sloužící ke studiu biblických textů. Její doména se omezuje pouze na samotné procházení textů Bible a neimplementuje funkcionality jako modlitení skupiny, nebo žaltář a další modlitby. Aplikace na mnoha místech nabízí uživateli placenou verzi, což působí rušivě a není v souladu s mými požadavky.

2.3 Beblia.com

Beblia.com je další webová aplikace, která nabízí čtečku Biblických textů. Obsahuje také velké množství překlaů a v rámci projektu je k dispozici i databáze různých překladů v XML formátu. Texty však nemají přiloženou žádnou licenci, takže není možné je volně používat bez souhlasu autora.

3 Návrh a struktura

Projekt je strukturován do monorepozitáře, který obsahuje několik balíčků (packages), do kterých je funkcionalita rozdělena. Kořenový balíček monorepozitáře obsahuje společnou konfiguraci nástrojů a společné balíčky.

Pro správu jsem použil balíčkovací nástroj pnpm pro node.js, který je alternativou k npm. Pro projekty s větším množstvím závislostí poskytuje značně lepší výkon a také možnost používat workspaces. Workspaces umožňují mít v rámci projektu více balíčků s vlastní konfigurací a vlastními závislostmi.

3.1 Balíčky

3.1.1 Kořenový balíček

Kořenový balíček má nastarosti uchovávání konfigurace

3.1.2 Backend

Sdílené soubory

Aplikace na backendu má modulární monolitickou architekturu, která zajišťuje rozdělení funkcionalit do skupin podle jejich domény a tím usnadňuje i kontrolu závislostí a umožňuje snadnější rozšířitelnost.

Jádrem aplikace je sada globálně dostupných nástrojů, které implementují funkcionality, které jsou nezávislé na doméně a používají se na více místech napříč aplikací.

Patří sem například middleware, který zajišťuje valitaci dat, ošetření chyb a logování. Dále také sdílené knihovny, jako například databázový adaptér.

Moduly

3.1.2.1 Struktura modulu

Moduly sdružují funkcionalitu, která je spojena na úrovni domény projektu. Každý modul obsahuje:

- **router:** mapuje HTTP požadavky na příslušné metody controlleru
- **controller:** Validuje a zpracovává data a požadavky. Zajišťuje komunikaci s routerem a spouští service
- **service:** zajišťuje získání a zpracování dat na základě požadavku. Výsledek vrací controlleru. Zajišťuje komunikaci mezi jednotlivými použitými závislostmi. Uchovává závislosti
- **repository:** zajišťuje komunikaci s databází prostřednictvím adaptéru a poskytuje metody specifické pro danou doménu. Provádí překlad mezi doménovými typy a typy pro elasticsearch klienta. Zajišťuje kontrolu a validaci dat. Obstarává konzistentnost dat v databázi.

3.1.3 Frontend

3.1.4 Databáze

3.2 Nastavení projektu

Popis Základní struktury projektu, konfigurace a popis složek a souborů v kořenovém balíčku.

3.3 Verzování

Pro verzování projektu jsem požil GIT a jako vzdálené úložiště GitHub. Nástroj GIT je velmi populární a v současnosti má velmi málo použitelných alternativ.

Pro vhodné použití s CI/CD pipeline, o kterém budu psát v další kapitole textu, a z důvodu lepší přehlednosti současného vývoje více funkcionalit, jsem zavedl také konvence pro pojmenování commitů a větví. Názvy mají pevnou strukturu a přidávají entitám verzovacího systému přesnější implicitní význam. Této struktury bude v budoucnu možné využít i pro automatizaci sémantického verzování.

Níže budu popisovat jednotlivé konvence.

3.3.1 Větve

Větev obsahuje větší množství ucelených změn, které mají společný účel. Větev může být používána dlouhou dobu. Může sloužit například pro vývoj celého konkrétního modulu, pro dokumentaci, úpravy infrastruktury, nebo refactoring balíčku.

Zvláštní roli mají větve dev a main, které jsou chráněny před přímým přidáváním commitů. Změny do nich lze přidávat jen pomocí pull-requestů. Větev dev slouží ke kompletaci změn z ostatních větví. Změny, které jsou přidány do větve main jsou určeny k vydání. Vydání nové verze se provede automaticky po oštítkování commitu.

Každá větev je pojmenována ve formátu $< typ > / < název >$, kde $< název >$ popisuje společný účel změn ve větvi (např. server-authentication). $< typ >$ přibližuje, jaký technický charakter změny mají (správa infrastruktury, dokumentace, testování...)

Povolené typy větví jsou následující:

- **chore:** Slouží pro údržbu projektu, závislostí, infrastruktury. Nepřidává další funkcionalitu, ani neopravuje chyby v kódu
- **feature:** Přidává skupinu funkcionalit do projektu (čtečka Bible, modlitební skupiny, ...)
- **release:** Stará se o poslední změny před vydáním nové verze. Typicky se zde provádí přidání change logu a zvýšení čísla verze.
- **docs:** Provádí se zde přidání a údržba přiložené dokumentace projektu.

3.3.2 Commity

Jednotlivé commity obsahují menší ucelené změny. Jeden commit by neměl obsahovat více změn, které spolu přímo nesouvisí (tématicky, nebo umístěním). Toto se mi nepodařilo v začátcích vývoje dodržet ve všech případech. Některé commity kromě přidání funkcionalit obsahují také refactoring, nebo opravu jiné části balíčku, která s primárními změnami přímo nesouvisí.

Pojmenování commitu je vždy ve formátu $< typ >:< popis >$, kde $< popis >$ stručně vysvětluje hlavní provedené změny v rámci daného commitu. Dále zpráva ke commitu obsahuje i podrobnější popis provedených změn.

Typy commitů jsou určeny pro budoucí implementaci automatického sémantického verzování. Usnadňují také orientaci ve změnách a upřesňují technický význam commitů. Povolené typy jsou tyto:

- **feature:** přidává novou funkcionalitu
- **fix:** opravuje chybu v kódu
- **test:** přidává automatizované testy pro již existující kód
- **docs:** upravuje dokumentaci
- **build:** změny v nastavení projektu, infrastruktuře a závislostch

3.4 CI/CD pipeline

GitHub jsem zvolil táké pro jeho rozšířenost a řadu možností, které poskytuje. mezi ně patří především služba GitHub Actions, kterou jsem použil pro vytvoření CI/CD pipeline.

GitHub Actions je flexibilní nástroj, který umožňuje spouštět akce, jako reakci na události v repozitáři.

3.5 Styl kódu

Popis konvencí

4 Použité technologie

4.1 TypeScript

TypesCript jako hlavní jazyk, ve kterém jsem projekt psal, jsem zvolil pro jeho univerzalitu. Kompiluje se do javascriptu, který je vhodný pro použití na backendu i frontendu aplikace. Toto jsem na začátku vývoje považoval za velkou výhodu. Typescript také na rozdíl od JavaScriptu poskytuje statické typování a s ním i typovou kontrolu, což v mnoha případech značně zjednodušuje vývoj, poskytuje vhodnější automatické doplňování kódu a odhaluje řadu chyb, které by

v kódu jinak vznikaly. Na mnoha místech však také kód značně znepřehledňuje a plní zbytečným přetypováním, které nemá vliv na výsledný běh kódu, vzhledem ke skutečnosti, že se vše kompiluje do javascriptu, který statické typování neimplementje. Proto je také všechny vstupy, které se do aplikace přijmou za běhu (data z databáze, HTTP požadavky, vstupy uživatele, ...), validovat znovu při běhu (k tomu jsem použil knihovnu Zod.)

4.2 Express

4.3 Vue.js

4.4 Elastic search

4.5 Docker

4.6 Github actions